DevOps Troubleshooting — 3–4 YOE Scenario-Based Interview Q&A

1. Kubernetes Pod is running, but application is returning 500 errors. What will you check?
Answer:
First, I check the application logs to understand why the application is returning 500 errors. Then I check whether the issue is related to application configuration, database connectivity, external dependencies, or environment variables.
I also check Pod events and recent deployments to identify whether any configuration or image change caused the issue.
After fixing the root cause, I verify the application endpoint again and monitor the Pod.
Simple ga gurthupettuko:
Pod Running unna app 500 isthe → logs chudu → config/database/dependency check cheyyi → recent changes compare cheyyi → fix chesi endpoint verify cheyyi.
Eng: If the Pod is running but the application returns 500, I check application logs, configuration, dependencies, and recent changes, then fix and verify the endpoint.
Example:
kubectl get pods -n <namespace> → Pod actually Running lo undho chudu.
kubectl logs <pod> -n <namespace> --tail=100 → Application 500 error reason chudu.
kubectl describe pod <pod> -n <namespace> → Pod events and configuration check cheyyi.
kubectl exec -it <pod> -n <namespace> -- env → Required environment variables available ga unnaya verify cheyyi.
kubectl exec -it <pod> -n <namespace> -- curl -v http://<dependency>:<port> → Dependency connectivity check cheyyi.
Issue identify chesi → configuration/dependency fix cheyyi → endpoint ni malli test chesi → 500 errors resolve ayyayo verify cheyyi.

---

2. Kubernetes Pod is continuously getting OOMKilled. What will you check?
Answer:
First, I check the Pod status and events to confirm that the container is being killed because it exceeded its memory limit.
Then I check the container's memory usage and compare it with the configured requests and limits.
I also investigate whether the application has a memory leak or whether the workload has increased recently.
Based on the root cause, I fix the application or adjust resources appropriately and monitor the Pod again.
Simple ga gurthupettuko:
OOMKilled ayithe → events chudu → memory usage check cheyyi → limit tho compare cheyyi → memory leak/workload check cheyyi → fix and monitor.
Eng: If a Pod is OOMKilled, I confirm the memory limit issue, check actual usage, investigate the application, and fix the resource or application problem.
Example:
kubectl get pod <pod> -n <namespace> → Pod status and restart count chudu.
kubectl describe pod <pod> -n <namespace> → OOMKilled reason/events verify cheyyi.
kubectl top pod <pod> -n <namespace> → Current memory usage check cheyyi.
kubectl get pod <pod> -n <namespace> -o yaml → Memory requests and limits verify cheyyi.
kubectl logs <pod> -n <namespace> --previous → Previous container logs chudu.
Issue application memory leak ayithe → application fix cheyyi; limit too low ayithe → appropriate memory limit increase chesi → rollout and Pod stability verify cheyyi.

---

3. Kubernetes Pod is running but keeps restarting. What will you investigate?
Answer:
First, I check the restart count and the reason for the previous container termination.
Then I inspect the previous container logs, Pod events, probes, exit code, and application configuration.
I also verify whether the container is being killed by a failed liveness probe, application crash, resource issue, or dependency failure.
After identifying the cause, I fix it and verify that the restart count stops increasing.
Simple ga gurthupettuko:
Pod Running but restarts avutunte → restart count chudu → previous logs → events → probes → exit code check cheyyi → root cause fix cheyyi.
Eng: I check the restart reason, previous logs, events, probes, and exit code to determine why the container keeps restarting.
Example:
kubectl get pods -n <namespace> → Restart count increase avutunda chudu.
kubectl describe pod <pod> -n <namespace> → Events and termination reason check cheyyi.
kubectl logs <pod> -n <namespace> --previous → Previous crashed container logs chudu.
kubectl get pod <pod> -n <namespace> -o jsonpath='{.status.containerStatuses[*].lastState.terminated.exitCode}' → Exit code check cheyyi.
kubectl get pod <pod> -n <namespace> -o yaml → Liveness/readiness probe configuration verify cheyyi.
Root cause fix chesi → kubectl get pod <pod> -n <namespace> → Restart count stable ga undho verify cheyyi.

---

4. Kubernetes Pod is stuck in Pending state even though nodes are available. What will you check?
Answer:
First, I describe the Pod and check scheduler events to understand why it cannot be scheduled.
Then I check CPU and memory requests, node labels, taints and tolerations, affinity rules, and resource availability.
A node being available does not always mean it can satisfy the Pod's scheduling requirements.
After correcting the scheduling constraint or resource requirement, I verify that the Pod gets scheduled successfully.
Simple ga gurthupettuko:
Pending ayithe → describe chudu → scheduler event identify cheyyi → resources/taints/labels/affinity check cheyyi → constraint fix cheyyi.
Eng: For a Pending Pod, I check scheduler events, resource requests, taints, labels, affinity, and node capacity.

Example:
kubectl get pods -n <namespace> → Pod Pending lo undho confirm cheyyi.
kubectl describe pod <pod> -n <namespace> → FailedScheduling events chudu.
kubectl get nodes --show-labels → Node labels verify cheyyi.
kubectl describe nodes → Node taints and allocatable resources check cheyyi.
kubectl top nodes → Current CPU/memory usage chudu.
Pod requests/affinity/taints configuration correct chesi → kubectl get pod <pod> -n <namespace> -w → Pod successfully scheduled ayyindo verify cheyyi.

---

5. Kubernetes Service has no Endpoints. What will you check?
Answer:
First, I check whether the backend Pods are running and ready.
Then I compare the Service selector with the labels on the Pods because a selector mismatch can result in no endpoints.
I also check whether readiness probes are failing because only ready Pods are normally added as Service endpoints.
After correcting the selector or Pod readiness issue, I verify that endpoints are populated.
Simple ga gurthupettuko:
Service endpoints empty ayithe → Pods Running/Ready aa chudu → Service selector and Pod labels compare cheyyi → readiness probe check cheyyi → endpoints verify cheyyi.
Eng: If a Service has no endpoints, I check Pod readiness and verify that the Service selector matches the Pod labels.
Example:
kubectl get svc <service> -n <namespace> → Service details chudu.
kubectl get endpoints <service> -n <namespace> → Endpoints empty ga unnaya check cheyyi.
kubectl get pods -n <namespace> --show-labels → Pod labels chudu.
kubectl describe svc <service> -n <namespace> → Service selector verify cheyyi.
kubectl describe pod <pod> -n <namespace> → Readiness probe failure check cheyyi.
Selector/labels or readiness issue fix chesi → kubectl get endpoints <service> -n <namespace> → Backend IPs appear ayyayo verify cheyyi.

---

6. Kubernetes application works inside the Pod but not through the Service. What will you check?
Answer:
First, I test the application directly inside the Pod to confirm that the application itself is listening on the expected port.
Then I check the Service port, targetPort, selector, and endpoints.
I also verify whether the application is listening on the correct network interface and whether a NetworkPolicy is blocking traffic.
After fixing the Service or networking configuration, I test access through the Service again.
Simple ga gurthupettuko:
Pod lopala app work ayithe → app problem kaadu → Service port/targetPort/selector/endpoints → NetworkPolicy check cheyyi → Service through test cheyyi.
Eng: If the application works inside the Pod but not through the Service, I verify the Service configuration, endpoints, ports, and network policies.
Example:
kubectl exec -it <pod> -n <namespace> -- curl -v http://localhost:<port> → Application directly reachable aa chudu.
kubectl get svc <service> -n <namespace> -o yaml → port and targetPort verify cheyyi.
kubectl get endpoints <service> -n <namespace> → Correct Pod IPs unnaya chudu.
kubectl get pods -n <namespace> --show-labels → Selector match avutunda verify cheyyi.
kubectl get networkpolicy -n <namespace> → NetworkPolicy restrictions check cheyyi.
Issue fix chesi → kubectl exec nunchi Service DNS/port ki curl chesi → response successfully vastunda verify cheyyi.

---

7. Kubernetes application is accessible inside the cluster but not from outside. What will you check?
Answer:
First, I confirm that the application and Service work correctly inside the cluster.
Then I troubleshoot the external traffic path such as Ingress, LoadBalancer, DNS, listener configuration, and firewall or security rules.
I also check whether the Ingress controller is healthy and whether the hostname is resolving to the correct endpoint.
After fixing the external routing issue, I test the application from outside the cluster.
Simple ga gurthupettuko:
Cluster lopala work ayithe → external path check cheyyi → Ingress/LB/DNS/firewall check cheyyi → outside nunchi test cheyyi.
Eng: If the application works inside the cluster but not externally, I trace the external traffic through Ingress or LoadBalancer, DNS, and network controls.
Example:
kubectl get svc -n <namespace> → Service type and external address chudu.
kubectl get ingress -n <namespace> → Ingress hostname/address verify cheyyi.
kubectl describe ingress <ingress> -n <namespace> → Routing/events check cheyyi.
kubectl get pods -n <ingress-namespace> → Ingress controller healthy ga undho chudu.
nslookup <hostname> → DNS correct IP ki resolve avutunda check cheyyi.
curl -vk https://<hostname> → External response and TLS/routing error chudu.
Issue DNS/Ingress/LB/firewall lo unte → fix chesi → external curl tho application accessible ga undho verify cheyyi.

8. Kubernetes Pods cannot pull an image from the private registry. What will you check?
Answer:
First, I check the Pod events because Kubernetes usually reports whether the issue is authentication, image name, tag, or registry connectivity.
Then I verify the image repository and tag and check whether the required image actually exists.
For a private registry, I also verify the image pull secret and whether the ServiceAccount is configured to use it.
After fixing authentication or the image reference, I recreate or restart the Pod and verify the image pulls successfully.
Simple ga gurthupettuko:
ImagePull fail ayithe → events chudu → image repo/tag check cheyyi → private registry secret verify cheyyi → fix chesi Pod pull verify cheyyi.
Eng: I check Pod events, image name and tag, registry availability, and image pull credentials before fixing the issue.
Example:
kubectl describe pod <pod> -n <namespace> → ImagePullBackOff or authentication error chudu.
kubectl get pod <pod> -n <namespace> -o jsonpath='{.spec.containers[*].image}' → Exact image/tag verify cheyyi.
kubectl get secret -n <namespace> → Image pull secret exists aa chudu.
kubectl get serviceaccount <serviceaccount> -n <namespace> -o yaml → imagePullSecrets configured aa verify cheyyi.
docker pull <registry>/<image>:<tag> → Registry image accessible aa independently check cheyyi.
Secret/image reference fix chesi → kubectl delete pod <pod> -n <namespace> → New Pod image successfully pull chesindo verify cheyyi.

---

9. Kubernetes readiness probe is failing, but the application process looks healthy. What will you check?
Answer:
First, I check the exact readiness probe failure message and verify the configured path, port, protocol, and timing.
Then I test the same endpoint directly from inside the Pod.
I also check whether the application takes longer to start than the configured initial delay or whether the probe is pointing to the wrong endpoint.
After correcting the probe configuration or application endpoint, I verify that the Pod becomes Ready.
Simple ga gurthupettuko:
Readiness fail ayithe → probe error chudu → path/port/protocol check cheyyi → same endpoint Pod lopala test cheyyi → timing correct cheyyi.
Eng: I verify the probe configuration and test the endpoint directly inside the Pod to determine whether the issue is the probe or the application.
Example:
kubectl describe pod <pod> -n <namespace> → Readiness probe failure message chudu.
kubectl get pod <pod> -n <namespace> -o yaml → Probe path, port, protocol, delay verify cheyyi.
kubectl exec -it <pod> -n <namespace> -- curl -v http://localhost:<port>/<path> → Probe endpoint manually test cheyyi.
kubectl logs <pod> -n <namespace> → Application startup timing/logs chudu.
kubectl get pod <pod> -n <namespace> -w → Ready status observe cheyyi.
Wrong path/port/timing ayithe → probe configuration fix chesi → rollout chesi → Pod Ready ga undho verify cheyyi.

---

10. Kubernetes node is showing NotReady. What will you investigate?
Answer:
First, I check the node conditions and events to identify why Kubernetes marked the node as NotReady.
Then I verify kubelet health, disk pressure, memory pressure, CPU, network connectivity, and container runtime status.
I also check whether the node has recently rebooted or whether the kubelet or container runtime stopped.
After fixing the node-level issue, I verify that the node returns to Ready state and that workloads are healthy.
Simple ga gurthupettuko:
Node NotReady ayithe → node conditions chudu → kubelet/runtime → disk/memory/network check cheyyi → issue fix chesi Ready verify cheyyi.
Eng: For a NotReady node, I check node conditions, kubelet, container runtime, resources, and network connectivity, then restore the node and verify workloads.
Example:
kubectl get nodes → Which node NotReady lo undo identify cheyyi.
kubectl describe node <node> → Conditions, events, pressure reasons chudu.
systemctl status kubelet → Kubelet running lo undho check cheyyi.
journalctl -u kubelet -n 100 --no-pager → Kubelet errors/logs chudu.
systemctl status containerd → Container runtime healthy ga undho verify cheyyi.
df -h → Disk full/disk pressure check cheyyi.
free -m → Memory pressure check cheyyi.
systemctl restart kubelet → Confirmed kubelet issue ayithe restart chesi → kubectl get nodes → Node Ready ki vachindo verify cheyyi.

---

11. Kubernetes Pod is stuck in Pending **state. What will you check?**
Answer:
First, I check the Pod status and describe the Pod to understand why Kubernetes is not scheduling it. Then I check node availability, CPU and memory resources, taints, tolerations, node selectors, affinity rules, and PVC requirements.
I also verify whether the Pod is requesting more resources than the available nodes can provide. If the issue is related to scheduling or resources, I fix the configuration or add suitable capacity and then verify that the Pod gets scheduled successfully.**Simple ga gurthupettuko:**
Pod Pending lo unte → describe chudu → scheduling reason identify cheyyi → nodes/resources/taints/PVC check cheyyi → issue fix chesi Pod schedule ayyindo verify cheyyi.

Eng: If a Pod is Pending, check its events and scheduler reason, then verify node resources, taints, affinity, and storage requirements before fixing the issue.
Example:
kubectl get pods -n <namespace> → Pod Pending lo undho confirm cheyyi.
kubectl describe pod <pod-name> -n <namespace> → Events section lo scheduling failure reason chudu.
kubectl get nodes → Available nodes unnaya check cheyyi.
kubectl describe nodes → CPU, memory, taints, and allocatable resources verify cheyyi.
kubectl get pvc -n <namespace> → Required PVC bound ayyinda check cheyyi.
Issue resource shortage ayithe → resource requests reduce cheyyi or node capacity increase cheyyi → kubectl get pods -n <namespace> tho Pod Running ayyindo verify cheyyi.

---

12. Kubernetes Pod is stuck in ContainerCreating **state. What will you check?**
Answer:
First, I describe the Pod and check the events because ContainerCreating usually indicates that Kubernetes is still preparing the container. I check volume mounting, image pulling, networking, Secrets, ConfigMaps, and container runtime errors.
I also verify whether the required volume or Secret exists and whether the node itself is healthy. After identifying the dependency or runtime issue, I fix it and verify that the container starts successfully.
Simple ga gurthupettuko:
ContainerCreating lo unte → Pod events chudu → volume/image/network/Secret/ConfigMap check cheyyi → node health verify cheyyi → issue fix chesi container start ayyindo chudu.
Eng: If a Pod remains in ContainerCreating, check Pod events and verify image pulling, volumes, Secrets, networking, and node health.
Example:
kubectl get pod <pod-name> -n <namespace> → Current Pod status chudu.
kubectl describe pod <pod-name> -n <namespace> → Events lo exact reason chudu.
kubectl get pvc -n <namespace> → Required volume available and bound undho check cheyyi.
kubectl get secret -n <namespace> → Required Secret exist avtunda verify cheyyi.
kubectl get nodes → Pod run avutunna node healthy ga undho check cheyyi.
kubectl describe node <node-name> → Node conditions and resource issues chudu.
Issue volume/image/runtime ayithe → root cause fix chesi → kubectl get pod <pod-name> -n <namespace> → Pod Running ayyindo verify cheyyi.

---

13. Kubernetes Pod is continuously restarting. What will you check?
Answer:
First, I check the Pod restart count, current state, previous container logs, and Pod events. Then I identify whether the application is crashing, failing its probes, running out of memory, or receiving an incorrect configuration.
I specifically check the previous container logs because the current container may already have restarted. After identifying the root cause, I fix the application or Kubernetes configuration and monitor the Pod to confirm that restarts stop.
Simple ga gurthupettuko:
Pod repeated ga restart ayithe → restart count chudu → previous logs check cheyyi → probes/config/OOM/application crash identify cheyyi → fix chesi monitor cheyyi.
Eng: If a Pod keeps restarting, check restart count, previous logs, events, probes, and resource limits to identify the crash reason.
Example:
kubectl get pods -n <namespace> → Restart count entha undho chudu.
kubectl logs <pod-name> -n <namespace> --previous → Previous container crash mundu logs chudu.
kubectl describe pod <pod-name> -n <namespace> → Events and container state verify cheyyi.
kubectl top pod <pod-name> -n <namespace> → Resource usage check cheyyi.
kubectl get pod <pod-name> -n <namespace> -o yaml → Liveness/readiness probes and configuration verify cheyyi.
Issue application crash/OOM/probe ayithe → root cause fix chesi → kubectl get pod <pod-name> -n <namespace> -w → Restarting stop ayyindo verify cheyyi.

---

14. Kubernetes Service exists, but traffic is not reaching the Pods. What will you check?
Answer:
First, I verify that the Service selector matches the labels on the target Pods. Then I check whether the Service has endpoints and whether the target Pods are Ready.
I also verify the Service port, targetPort, containerPort, and network policies. If the Service has no endpoints, I focus on selectors and Pod readiness because Kubernetes cannot route traffic to Pods that are not selected or ready.
Simple ga gurthupettuko:
Service undi traffic lekapothe → selector check cheyyi → Endpoints chudu → Pods Ready aa check cheyyi → ports and network policy verify cheyyi.
Eng: If a Service is not routing traffic, verify selectors, endpoints, Pod readiness, ports, and network policies.
Example:
kubectl get svc -n <namespace> → Service and ports chudu.
kubectl describe svc <service-name> -n <namespace> → Selector and port configuration verify cheyyi.
kubectl get pods --show-labels -n <namespace> → Service selector ki Pod labels match avutunnaya chudu.
kubectl get endpoints <service-name> -n <namespace> → Backend endpoints unnaya check cheyyi.
kubectl get pods -n <namespace> → Target Pods Ready state lo unnaya verify cheyyi.
kubectl get networkpolicy -n <namespace> → Traffic ni NetworkPolicy block chestunda check cheyyi.
Selector/port issue ayithe → configuration fix chesi → kubectl get endpoints <service-name> -n <namespace> → endpoints populate ayyayo verify cheyyi.

15. Kubernetes Deployment rollout is stuck. What will you check?
Answer:
First, I check the Deployment rollout status and ReplicaSet status to understand where the rollout stopped. Then I check Pod states, events, image availability, readiness probes, resource limits, and scheduling issues.
I compare the new ReplicaSet with the previous working one to identify what changed. After fixing the problem, I continue or restart the rollout and verify that all desired replicas become Ready.
Simple ga gurthupettuko:
Rollout stuck ayithe → rollout status chudu → ReplicaSet/Pods check cheyyi → events/image/probes/resources verify cheyyi → issue fix chesi rollout complete cheyyi.
Eng: If a Deployment rollout is stuck, inspect rollout status, ReplicaSets, Pods, events, images, probes, and resources to find the blocking issue.
Example:
kubectl rollout status deployment/<deployment-name> -n <namespace> → Rollout ekkada stuck ayyindo chudu.
kubectl get rs -n <namespace> → New and old ReplicaSets compare cheyyi.
kubectl get pods -n <namespace> → New Pods status chudu.
kubectl describe pod <pod-name> -n <namespace> → Scheduling/probe/image errors chudu.
kubectl describe deployment <deployment-name> -n <namespace> → Deployment events verify cheyyi.
kubectl rollout history deployment/<deployment-name> -n <namespace> → Previous revisions chudu.
Issue fix ayithe → kubectl rollout status deployment/<deployment-name> -n <namespace> → Rollout successfully complete ayyindo verify cheyyi.

---

16. Docker container is running, but the application is not reachable. What will you check?
Answer:
First, I verify that the container is actually running and check its logs for application startup errors. Then I verify the application listening port, Docker port mapping, bind address, and network configuration.
I also check whether the application is listening only on localhost inside the container instead of 0.0.0.0. After correcting the port or application configuration, I test connectivity again.
Simple ga gurthupettuko:
Container running kani app access lekapothe → logs chudu → app listening port check cheyyi → Docker port mapping verify cheyyi → bind address check cheyyi → connectivity test cheyyi.
Eng: If the container is running but inaccessible, check application logs, listening ports, Docker port mapping, bind address, and network connectivity.
Example:
docker ps → Container running lo undho and port mapping enti chudu.
docker logs <container_id> → Application startup or binding errors chudu.
docker exec <container_id> ss -lntp → Container lopala application ye port meeda listen avutundo check cheyyi.
docker port <container_id> → Host-to-container port mapping verify cheyyi.
curl http://localhost:<host-port> → Application host nunchi reachable ga undho test cheyyi.
App 127.0.0.1 ki bind ayithe → application configuration lo 0.0.0.0 ki change cheyyi → container restart chesi connectivity verify cheyyi.

---

17. Docker image build suddenly fails after a Dockerfile change. What will you check?
Answer:
First, I check the Docker build output and identify the exact Dockerfile instruction where the failure occurs. Then I verify the changed instruction, build context, file paths, package installation, base image, and network access to package repositories.
I also compare the current Dockerfile with the last working version to identify the problematic change. After correcting it, I rebuild the image and verify that the image is created successfully.
Simple ga gurthupettuko:
Docker build fail ayithe → build error chudu → failed Dockerfile line identify cheyyi → path/package/base image/network check cheyyi → fix chesi rebuild cheyyi.
Eng: If a Docker build fails after a Dockerfile change, identify the failed instruction, compare it with the last working version, fix the root cause, and rebuild.
Example:
docker build -t myapp:test . → Build run chesi exact error ekkada vachindo chudu.
docker build --no-cache -t myapp:test . → Cache issue unda ani verify cheyyi.
git diff HEAD~1 Dockerfile → Recent Dockerfile changes chudu.
docker images → Base image available/build output verify cheyyi.
cat Dockerfile → Failed instruction, paths, package commands check cheyyi.
Package/network issue ayithe → repository/connectivity or Dockerfile dependency fix chesi → docker build -t myapp:test . → Image successfully build ayyindo verify cheyyi.

---

18. Ansible playbook is hanging on one task. What will you check?
Answer:
First, I identify the exact task where Ansible is waiting and check whether it is waiting for SSH, a command, user input, a package manager lock, or an application operation.
I also check the target server's CPU, memory, disk, network connectivity, and running processes. If the task is waiting because of an interactive command or lock, I correct the task or remove the blocking condition and rerun it.
Simple ga gurthupettuko:
Ansible task hang ayithe → exact task identify cheyyi → SSH/command/input/package lock check cheyyi → target server resources/processes chudu → block remove chesi rerun cheyyi.

Eng: If an Ansible task hangs, identify the waiting task and check SSH, commands, locks, interactive input, and target-server resources.
Example:
ansible-playbook -i inventory playbook.yml -vvv → Detailed output tho exact task ekkada hang ayindo chudu.
ansible <host> -i inventory -m ping → Target server SSH/connectivity verify cheyyi.
ps -ef → Target server lo stuck process unda chudu.
top → CPU/memory issue unda check cheyyi.
df -h → Disk full ayinda verify cheyyi.
ps aux | grep -E 'apt|yum|dnf' → Package manager lock/process unda chudu.
Interactive command ayithe → command ni non-interactive ga modify chesi → ansible-playbook -i inventory playbook.yml → Task complete ayyindo verify cheyyi.

---

19. Ansible playbook reports UNREACHABLE **for a server. What will you check?**
Answer:
First, I verify whether the target server is reachable over the network and whether SSH is working. Then I check the inventory hostname or IP, SSH port, username, private key, firewall rules, DNS resolution, and SSH configuration.
I also verify whether the Ansible user has permission to connect to the server. After fixing the connectivity or authentication issue, I test the connection with Ansible before rerunning the complete playbook.
Simple ga gurthupettuko:
UNREACHABLE ayithe → network check → SSH check → inventory/IP/port/user/key verify cheyyi → firewall/DNS check cheyyi → Ansible ping tho test cheyyi.
Eng: If Ansible reports UNREACHABLE, verify network connectivity, SSH, inventory, port, user, key, DNS, and firewall before rerunning the playbook.
Example:
ansible <host> -i inventory -m ping -vvv → Ansible connectivity and exact SSH error chudu.
ping <server-ip> → Basic network connectivity check cheyyi.
ssh -i <key> <user>@<server-ip> → Direct SSH login verify cheyyi.
nc -zv <server-ip> 22 → SSH port reachable ga undho check cheyyi.
getent hosts <hostname> → DNS resolution verify cheyyi.
ansible-inventory -i inventory --host <host> → Inventory lo IP/user/variables correct ga unnaya chudu.
Issue SSH/firewall/inventory ayithe → fix chesi → ansible <host> -i inventory -m ping → SUCCESS vachindo verify cheyyi.

---

20. Jenkins pipeline is successful, but the deployed application is showing the previous version. What will you investigate?
Answer:
First, I trace the version from the Git commit through the Jenkins build, Docker image or artifact, deployment configuration, and finally the running application.
Then I verify whether Jenkins actually built and published the expected version and whether the deployment picked that exact version. I also check image tags, registry contents, rollout status, and whether old Pods or containers are still serving traffic.
After identifying where the old version was introduced, I fix the pipeline or deployment configuration and verify the running version.
Simple ga gurthupettuko:
Jenkins success kani old app vasthe → Git commit nunchi running app varaku version trace cheyyi → image/artifact/tag/deployment check cheyyi → old Pod/container traffic lo unda chudu → fix chesi verify cheyyi.
Eng: If Jenkins succeeds but the old application version is running, trace the version from Git to Jenkins, registry, deployment, and running Pods or containers.
Example:
git rev-parse HEAD → Jenkins build ye commit meeda run ayindo identify cheyyi.
docker images → Expected image/tag build ayyinda check cheyyi.
docker inspect <container_id> --format='{{.Config.Image}}' → Running container ye image use chestundo chudu.
kubectl get deployment <deployment-name> -n <namespace> -o=jsonpath='{.spec.template.spec.containers[*].image}' → Kubernetes Deployment lo expected image tag undho verify cheyyi.
kubectl get pods -n <namespace> -o wide → Old Pods still running unnaya check cheyyi.
kubectl rollout status deployment/<deployment-name> -n <namespace> → Latest rollout complete ayyinda verify cheyyi.
Issue wrong tag/old image/rollout ayithe → pipeline or deployment config fix chesi → correct version redeploy cheyyi → running application version verify cheyyi.

---

21. Kubernetes Pod is in CrashLoopBackOff**. What will you check?**
Answer:
First, I check the Pod status, restart count, and container logs to understand why the application is repeatedly crashing. Then I check the previous container logs, Pod events, environment variables, Secrets, ConfigMaps, probes, and resource limits.
I also verify whether the application command or arguments are correct. After identifying the crash reason, I fix the application or Kubernetes configuration and monitor the Pod to confirm that it becomes stable.
Simple ga gurthupettuko:
CrashLoopBackOff → logs chudu → previous logs/events check cheyyi → config/probes/resources/command verify cheyyi → root cause fix chesi Pod stable ayyindo chudu.
Eng: If a Pod is in CrashLoopBackOff, check current and previous logs, events, configuration, probes, resources, and startup commands.
Example:
kubectl get pods -n <namespace> → Pod restart count and status chudu.
kubectl logs <pod-name> -n <namespace> → Current container error chudu.
kubectl logs <pod-name> -n <namespace> --previous → Previous crashed container logs chudu.
kubectl describe pod <pod-name> -n <namespace> → Events and termination reason check cheyyi.

kubectl get pod <pod-name> -n <namespace> -o yaml → Command, args, environment, and probes verify cheyyi.
kubectl top pod <pod-name> -n <namespace> → Resource usage check cheyyi.
Issue config/OOM/probe ayithe → root cause fix chesi → kubectl get pod <pod-name> -n <namespace> -w → Restarting stop ayi Running/Ready ayyindo verify cheyyi.

---

22. Kubernetes Pod is showing ImagePullBackOff**. What will you check?**
Answer:
First, I check the exact image name and tag configured in the Pod or Deployment. Then I verify whether that image exists in the container registry and whether the cluster has permission to pull it.
I also check image pull Secrets, registry connectivity, DNS, and the Pod events for the exact pull error. After correcting the image, credentials, or registry access, I restart or update the workload and verify that the image is pulled successfully.
Simple ga gurthupettuko:
ImagePullBackOff → image name/tag check → registry lo image undha chudu → pull Secret/permissions/network verify cheyyi → fix chesi Pod pull ayyindo verify cheyyi.
Eng: If a Pod shows ImagePullBackOff, verify the image tag, registry availability, pull credentials, connectivity, and Pod events.
Example:
kubectl describe pod <pod-name> -n <namespace> → Events lo image pull exact error chudu.
kubectl get deployment <deployment-name> -n <namespace> -o yaml → Image repository and tag verify cheyyi.
kubectl get secrets -n <namespace> → Image pull Secret available ga undho check cheyyi.
kubectl get pod <pod-name> -n <namespace> -o jsonpath='{.spec.imagePullSecrets}' → Correct pull Secret configured undho chudu.
docker pull <registry>/<image>:<tag> → Registry lo image/tag valid ga undho test cheyyi.
Wrong tag or credentials ayithe → correct image/Secret configure chesi → kubectl rollout restart deployment/<deployment-name> -n <namespace> → Pod image successfully pull ayyindo verify cheyyi.

---

23. Kubernetes Pod is Running **but not** Ready**. What will you check?**
Answer:
First, I check the readiness probe configuration and Pod events. A Pod can be running while the application inside it is not ready to receive traffic.
I verify the application port, health endpoint, startup time, dependencies, environment variables, and readiness probe settings. After correcting the application or probe configuration, I verify that the Pod becomes Ready and receives traffic.
Simple ga gurthupettuko:
Pod Running kani Ready kaakapothe → readiness probe check → health endpoint/port/config/dependencies verify cheyyi → fix chesi Ready ayyindo chudu.
Eng: If a Pod is Running but not Ready, check the readiness probe, application health endpoint, port, configuration, and dependencies.
Example:
kubectl get pods -n <namespace> → READY column lo Pod ready kaadho chudu.
kubectl describe pod <pod-name> -n <namespace> → Readiness probe failures and events chudu.
kubectl get pod <pod-name> -n <namespace> -o yaml → Probe path, port, and timing verify cheyyi.
kubectl exec <pod-name> -n <namespace> -- curl -v http://localhost:<port>/health → Application health endpoint test cheyyi.
kubectl logs <pod-name> -n <namespace> → Application startup/health errors chudu.
Probe/config issue ayithe → fix chesi → kubectl get pods -n <namespace> -w → Pod 1/1 Ready ayyindo verify cheyyi.

---

24. Kubernetes Ingress is configured, but the application is not accessible. What will you check?
Answer:
First, I check whether the Ingress resource is created correctly and whether the Ingress Controller is healthy. Then I verify the hostname, TLS configuration, backend Service, Service port, endpoints, and DNS resolution.
I also check Ingress Controller logs and whether the request is reaching the correct backend. After identifying the routing, DNS, TLS, or Service issue, I fix it and test the application again.
Simple ga gurthupettuko:
Ingress access lekapothe → Ingress Controller check → host/DNS/TLS verify → backend Service/port/endpoints check → logs chusi fix cheyyi.
Eng: If an Ingress is not accessible, verify the Ingress Controller, DNS, hostname, TLS, backend Service, ports, and endpoints.
Example:
kubectl get ingress -n <namespace> → Ingress address and rules chudu.
kubectl describe ingress <ingress-name> -n <namespace> → Host, backend, TLS, and events verify cheyyi.
kubectl get pods -n <ingress-namespace> → Ingress Controller healthy ga undho check cheyyi.
kubectl logs <ingress-controller-pod> -n <ingress-namespace> → Routing errors chudu.
nslookup <hostname> → DNS correct IP ki resolve avutunda verify cheyyi.
kubectl get endpoints <service-name> -n <namespace> → Backend Pods available unnaya chudu.
Issue DNS/TLS/routing ayithe → fix chesi → curl -vk https://<hostname> → Application accessible ga undho verify cheyyi.

---

25. Kubernetes node suddenly becomes NotReady**. What will you check?**
Answer:
First, I check the node conditions and recent events to understand why Kubernetes marked it NotReady. Then I check kubelet status, container runtime, CPU, memory, disk space, network connectivity, and system health.
I also check whether the node has disk pressure, memory pressure, or a runtime failure. After fixing the node-level issue, I verify that kubelet becomes healthy and the node returns to Ready state.

Simple ga gurthupettuko:
Node NotReady → describe node → conditions/events chudu → kubelet/runtime/resources/network check cheyyi → issue fix chesi Ready verify cheyyi.
Eng: If a Kubernetes node becomes NotReady, check node conditions, kubelet, container runtime, resources, disk, and network health.
Example:
kubectl get nodes → Which node NotReady undo identify cheyyi.
kubectl describe node <node-name> → Conditions and events chudu.
systemctl status kubelet → Kubelet running lo undho check cheyyi.
journalctl -u kubelet -n 100 --no-pager → Kubelet errors chudu.
systemctl status containerd → Container runtime healthy ga undho verify cheyyi.
df -h → Disk pressure unda check cheyyi.
free -m → Memory issue unda chudu.
Issue kubelet/runtime/disk ayithe → fix chesi → kubectl get nodes → Node Ready ayyindo verify cheyyi.

---

26. Linux server disk usage suddenly reaches 100%. What will you do?
Answer:
First, I confirm which filesystem is full and identify the directories or files consuming the most space. Then I check logs, application data, temporary files, deleted files still held by processes, and Docker data if Docker is installed.
I safely remove or archive unnecessary data and verify that the application has enough disk space. I also identify why disk usage increased and configure log rotation or retention to prevent the issue from happening again.
Simple ga gurthupettuko:
Disk 100% → df chudu → du tho ekkuva space ekkada undo find cheyyi → logs/temp/Docker data check → safely clean → future retention set cheyyi.
Eng: If disk usage reaches 100%, identify the full filesystem and largest consumers, safely clean unnecessary data, and prevent recurrence.
Example:
df -h → Which filesystem full undo chudu.
du -sh /* 2>/dev/null | sort -h → Large directories identify cheyyi.
du -sh /var/log/* 2>/dev/null → Logs ekkuva space tiskuntunnaya check cheyyi.
docker system df → Docker data entha space use chestundo chudu.
lsof +L1 → Deleted but still-open files identify cheyyi.
Unnecessary logs/files ayithe → safely clean/archive chesi → df -h → Free space verify cheyyi.

---

27. Linux server memory usage is very high. How will you troubleshoot?
Answer:
First, I check memory and swap usage and identify which processes are consuming the most memory. Then I check whether the usage is caused by an application, Java process, container, or memory leak.
I also review system logs and application behavior before restarting anything. After identifying the process or configuration causing the issue, I tune or restart it safely and monitor memory usage.
Simple ga gurthupettuko:
Memory high → free chudu → top/ps tho process identify cheyyi → application/container leak check → safe fix chesi monitor cheyyi.
Eng: If memory usage is high, identify the memory-consuming process, check for leaks or workload issues, fix safely, and monitor the server.
Example:
free -m → Total, used, available memory and swap usage chudu.
top → High-memory process identify cheyyi.
ps aux --sort=-%mem | head → Top memory-consuming processes chudu.
docker stats → Containers memory ekkuva use chestunnaya check cheyyi.
journalctl -k -n 100 --no-pager → Kernel/OOM-related messages chudu.
Memory leak or excessive workload ayithe → application/config fix or controlled restart cheyyi → free -m and top tho memory normal ayyindo verify cheyyi.

---

28. Linux server load average is very high. What will you check?
Answer:
First, I check CPU utilization, memory pressure, disk I/O, and the processes contributing to the load. High load does not always mean CPU is the only problem; blocked I/O can also increase load.
I identify the process or workload causing the pressure and determine whether it is expected or abnormal. Then I optimize, stop, or distribute the workload as appropriate and verify that the load decreases.
Simple ga gurthupettuko:
Load high → CPU matrame kaadu → CPU/memory/disk I/O check → high-load process identify → root cause fix → load monitor cheyyi.
Eng: If load average is high, check CPU, memory, disk I/O, and blocked processes to identify the real source of the load.
Example:
uptime → Load average entha undo chudu.
top → CPU-consuming and blocked processes identify cheyyi.
vmstat 1 5 → CPU, memory, and system wait activity check cheyyi.
iostat -xz 1 5 → Disk I/O bottleneck unda verify cheyyi.
ps -eo state,pid,cmd | grep '^D' → Uninterruptible I/O wait processes chudu.
Issue disk/CPU/workload ayithe → bottleneck fix or workload distribute chesi → uptime and top tho load normal avutunda monitor cheyyi.

29. Application is running on Linux, but the port is not reachable. What will you check?
Answer:
First, I check whether the application process is running and whether it is listening on the expected port. Then I verify the bind address, local connectivity, firewall rules, security groups, and network path.
I test the port locally first and then from the client side to identify whether the problem is with the application or the network. After fixing the listening or network configuration, I test connectivity again.
Simple ga gurthupettuko:
Port access lekapothe → process/listening port check → local curl/nc test → bind address/firewall/security group/network path check → fix and retest.
Eng: If a port is unreachable, verify the application listener first, then check bind address, firewall, security groups, and network connectivity.
Example:
ss -lntp | grep :8080 → Application expected port meeda listen avutunda chudu.
ps -ef | grep <process> → Application process running lo undho verify cheyyi.
curl http://localhost:8080 → Server lopala application reachable ga undho test cheyyi.
nc -zv <server-ip> 8080 → Port connectivity test cheyyi.
sudo firewall-cmd --list-all → Firewall rules check cheyyi.
sudo iptables -L -n → Existing firewall rules verify cheyyi.
Application localhost ki matrame bind ayithe → bind address 0.0.0.0 ki correct cheyyi → firewall/network fix chesi → nc/curl tho external access verify cheyyi.

---

30. Application is returning HTTP 502 Bad Gateway. What will you check?
Answer:
First, I identify which component is returning the 502, such as Nginx, Ingress, or a load balancer. Then I check whether the backend application is running and whether the proxy can connect to the correct backend port.
I also verify backend health, Service endpoints, DNS, connection timeouts, and proxy configuration. After identifying the broken connection between the proxy and backend, I fix it and test the request again.
Simple ga gurthupettuko:
502 → proxy evaru return chestundo identify → backend app running aa chudu → backend port/endpoints/DNS check → proxy config fix → request retest.
Eng: For a 502 error, identify the proxy or gateway, verify backend health and connectivity, check ports and configuration, and fix the broken upstream connection.
Example:
curl -I https://<hostname> → HTTP response and 502 confirm cheyyi.
systemctl status nginx → Nginx running lo undho check cheyyi.
tail -n 100 /var/log/nginx/error.log → Upstream connection error chudu.
curl http://<backend-ip>:<port> → Backend directly reachable ga undho test cheyyi.
kubectl get endpoints <service-name> -n <namespace> → Kubernetes backend endpoints unnaya chudu.
kubectl logs <backend-pod> -n <namespace> → Backend application errors check cheyyi.
Backend port/config issue ayithe → fix chesi → curl -I https://<hostname> → 200/expected response vachindo verify cheyyi.

---

31. Application is returning HTTP 504 Gateway Timeout. What will you check?
Answer:
First, I determine whether the timeout is happening between the client and proxy or between the proxy and backend application. Then I check backend response time, application logs, database calls, network latency, and proxy timeout settings.
I also verify whether the backend is overloaded or stuck on a slow dependency. After identifying the slow component, I fix the application or dependency and adjust timeout settings only when appropriate.
Simple ga gurthupettuko:
504 → evaru timeout istunnaro identify → backend response time check → DB/network/dependency check → proxy timeout verify → root cause fix.
Eng: For a 504 timeout, identify the slow upstream component and check application response time, dependencies, network latency, and timeout configuration.
Example:
curl -v https://<hostname> → Request ekkada delay avutundo chudu.
tail -n 100 /var/log/nginx/error.log → Upstream timeout details check cheyyi.
curl -w '%{time_total}\n' http://<backend>:<port>/health → Backend response time measure cheyyi.
top → Backend CPU/memory pressure check cheyyi.
kubectl logs <pod-name> -n <namespace> → Slow application/DB operation errors chudu.
kubectl describe ingress <ingress-name> -n <namespace> → Ingress configuration verify cheyyi.
Slow dependency ayithe → root cause optimize/fix chesi → timeout configuration required ayithe controlled ga adjust cheyyi → request malli test cheyyi.

---

32. Git push is rejected because of a conflict. What will you do?
Answer:
First, I check whether the remote branch contains commits that are not present locally. I fetch the latest remote changes and inspect the differences before integrating them.
I then rebase or merge the remote changes into my branch, resolve any conflicts carefully, test the code, and push again. I avoid overwriting remote history unless there is an approved reason to do so.
Simple ga gurthupettuko:
Push reject → remote changes fetch cheyyi → difference check → merge/rebase → conflicts resolve → test → push again.

Eng: If Git rejects a push because the remote has newer changes, fetch them, integrate them safely, resolve conflicts, test, and push again.

Example:

git status → Current branch and local changes chudu.
git fetch origin → Latest remote changes fetch cheyyi.
git log --oneline HEAD..origin/<branch> → Remote lo new commits unnaya check cheyyi.
git diff HEAD origin/<branch> → Local and remote differences chudu.
git pull --rebase origin <branch> → Remote changes ni safely rebase cheyyi.
git status → Conflicted files identify cheyyi.
git add <file> → Conflict resolve chesina file stage cheyyi.
git rebase --continue → Rebase complete cheyyi.
Tests pass ayyaka → git push origin <branch> → Push successful ayyindo verify cheyyi.

---

33. Git repository clone is failing in a CI/CD pipeline. What will you check?

Answer:

First, I check the exact Git error to determine whether the issue is authentication, repository access, DNS, network connectivity, or an incorrect repository URL.
Then I verify the CI/CD credentials, repository permissions, branch, SSH key or token, and connectivity from the build agent. After fixing the access or configuration issue, I test the clone again from the same CI environment.

Simple ga gurthupettuko:

Git clone fail → exact error chudu → URL/auth/network/permissions check → CI credential verify → same agent nunchi clone test cheyyi.

Eng: If Git clone fails in CI/CD, check the repository URL, credentials, permissions, network, DNS, and the build agent environment.

Example:

git clone <repo-url> → Exact clone error identify cheyyi.
git ls-remote <repo-url> → Repository access and authentication test cheyyi.
ssh -T git@<git-host> → SSH authentication verify cheyyi.
nslookup <git-host> → DNS resolution check cheyyi.
curl -I https://<git-host> → HTTPS connectivity verify cheyyi.
CI credential issue ayithe → Jenkins/GitHub Actions secret or credential correct ga configure chesi → same CI agent nunchi git ls-remote run chesi → repository access verify cheyyi.

---

34. Terraform apply **is failing after another engineer changed the infrastructure manually. What will you check?**

Answer:

First, I check the Terraform error and compare the Terraform state with the actual infrastructure. Manual changes can create drift between the state and the real resources.
I run a plan to understand what Terraform wants to change and determine whether the manual change should be preserved or reverted. After reconciling the state and configuration, I apply the approved change and verify the infrastructure.

Simple ga gurthupettuko:

Manual change tarvatha Terraform fail → error chudu → state vs actual infra compare → drift identify → config/state reconcile → plan → apply.

Eng: If Terraform fails after manual infrastructure changes, check for state drift, review the plan, reconcile the desired state, and apply safely.

Example:

terraform plan → Terraform actual infra tho state/config difference enti chudu.
terraform state list → Terraform manage chestunna resources identify cheyyi.
terraform state show <resource> → Resource state details verify cheyyi.
Cloud console/CLI → Actual resource configuration compare cheyyi.
terraform plan -out=tfplan → Proposed changes review cheyyi.
Manual change preserve cheyyali ante → Terraform configuration update or approved state reconciliation cheyyi → terraform apply tfplan → Final infrastructure state verify cheyyi.

---

35. Terraform plan shows many unexpected changes. What will you investigate?

Answer:

First, I do not apply the plan immediately. I inspect the resources and attributes Terraform wants to change and identify why they differ from the configuration or state.
I check provider versions, variables, remote state, manually changed infrastructure, generated values, and resource lifecycle settings. After finding the reason for the unexpected diff, I correct the configuration or state and run the plan again until the changes are understood.

Simple ga gurthupettuko:

Unexpected Terraform changes → immediate apply cheyyaku → plan details chudu → state/config/provider/variables/manual changes compare → difference fix → plan malli run cheyyi.

Eng: If Terraform shows unexpected changes, review the plan carefully, compare state with configuration and infrastructure, fix the difference, and re-plan before applying.

Example:

terraform plan → Unexpected resources/attributes identify cheyyi.
terraform show → Current state information review cheyyi.
terraform state list → Managed resources verify cheyyi.
terraform providers → Provider versions and dependencies check cheyyi.
terraform output → Current Terraform outputs verify cheyyi.

Cloud CLI/console → Actual resource settings compare cheyyi.
Issue config/provider/drift ayithe → fix chesi → terraform plan → Expected changes matrame unnaya verify cheyyi → tarvatha apply cheyyi.

---

36. Terraform state is locked and another deployment cannot run. What will you do?
Answer:
First, I verify whether another Terraform operation is genuinely running because the state lock protects against concurrent modifications. I do not remove the lock immediately.
If the previous operation has actually failed and left a stale lock, I identify the lock information and remove it only after confirming that no Terraform process is using the state. Then I rerun the operation safely.
Simple ga gurthupettuko:
State locked → first another Terraform run active aa check → active unte wait → stale lock ani confirm ayithe matrame unlock → plan/apply retry.
Eng: If Terraform state is locked, first confirm whether another operation is running; remove a stale lock only after confirming it is safe.
Example:
terraform plan → State lock error confirm cheyyi.
CI/CD system → Another Terraform job currently running undho check cheyyi.
ps -ef | grep terraform → Server lo active Terraform process undha chudu.
terraform force-unlock <LOCK_ID> → Stale lock ani confirm ayina tarvatha matrame remove cheyyi.
terraform plan → Lock release ayyaka plan safely run cheyyi.
terraform apply → Approved changes apply chesi final state verify cheyyi.

---

37. Docker container is consuming very high CPU or memory. What will you check?
Answer:
First, I identify the container consuming the resources and check its current CPU and memory usage. Then I inspect the application process, workload, logs, and whether the container has appropriate resource limits.
I also check for application loops, memory leaks, excessive requests, or unexpectedly high traffic. After identifying the root cause, I optimize the application or resource configuration and monitor the container again.
Simple ga gurthupettuko:
Docker resource high → container identify → docker stats → process/logs check → limits and workload verify → root cause fix → monitor.
Eng: If a Docker container consumes excessive resources, identify the container and process, check workload and limits, fix the cause, and monitor again.
Example:
docker stats → High CPU/memory container identify cheyyi.
docker top <container_id> → Container lopala resource-consuming processes chudu.
docker logs <container_id> → Application errors or abnormal loops check cheyyi.
docker inspect <container_id> → Resource limits and configuration verify cheyyi.
docker exec <container_id> top → Container lopala process usage chudu.
Application leak/high workload ayithe → application/config optimize chesi or suitable limits configure chesi → docker stats → Usage normal ayyindo verify cheyyi.

---

38. Helm upgrade succeeds, but the new configuration is not visible in the application. What will you check?
Answer:
First, I verify that Helm actually applied the expected values and that the generated Kubernetes manifest contains the new configuration. Then I check how the application consumes the configuration, whether through environment variables, ConfigMaps, or mounted files.
If the application reads configuration only during startup, the Pods may need to restart. I verify the running Pod configuration and restart the workload safely if required.
Simple ga gurthupettuko:
Helm success kani config reflect avvakapothe → Helm values/rendered manifest check → ConfigMap/Secret/POD consumption check → restart required aa chudu → verify.
Eng: If a Helm upgrade succeeds but configuration is not reflected, verify rendered values, Kubernetes configuration objects, Pod consumption, and restart requirements.
Example:
helm get values <release-name> -n <namespace> → Release ki actual values enti chudu.
helm get manifest <release-name> -n <namespace> → Generated Kubernetes manifest lo new config undho verify cheyyi.
kubectl get configmap <configmap-name> -n <namespace> -o yaml → ConfigMap updated ayyinda chudu.
kubectl get secret <secret-name> -n <namespace> -o yaml → Secret configuration verify cheyyi.
kubectl exec <pod-name> -n <namespace> -- env → Environment variable application ki correct ga vachinda check cheyyi.
Startup-time config ayithe → kubectl rollout restart deployment/<deployment-name> -n <namespace> → New configuration tho Pods restart ayyayo verify cheyyi.

---

39. CI/CD pipeline deployment succeeded, but users are receiving errors. What will you investigate?
Answer:
First, I assess the Production impact and identify whether all users or only a specific percentage are affected. Then I check deployment status, application logs, health checks, infrastructure metrics, dependencies, and recent configuration changes.
I compare the new version with the previous known-good version and determine whether the issue can be fixed safely or whether rollback is required. After recovery, I verify application health and monitor the system closely.

Simple ga gurthupettuko:
Deploy success kani users errors → impact assess → app logs/health/metrics/dependencies check → new vs old version compare → fix or rollback → monitor.
Eng: If deployment succeeds but users see errors, assess impact, inspect application and infrastructure health, compare versions, and recover safely.
Example:
kubectl get pods -n <namespace> → New Pods healthy ga unnaya chudu.
kubectl logs <pod-name> -n <namespace> → Application errors identify cheyyi.
kubectl get events -n <namespace> --sort-by=.lastTimestamp → Recent Kubernetes events chudu.
curl -I https://<application-url> → Application response verify cheyyi.
kubectl rollout history deployment/<deployment-name> -n <namespace> → Previous deployment revision identify cheyyi.
Issue new release valla ayithe → kubectl rollout undo deployment/<deployment-name> -n <namespace> → Known-good version ki rollback chesi → kubectl rollout status deployment/<deployment-name> -n <namespace> → Recovery verify cheyyi.

---

40. Production deployment is failing across multiple DevOps tools. How will you troubleshoot it end-to-end?
Answer:
First, I identify exactly where the deployment flow is failing instead of changing multiple components at once. I trace the request from Git to CI/CD, Docker image or artifact, registry, Helm or deployment configuration, Kubernetes resources, and finally the application.
At each stage, I check logs, configuration, credentials, connectivity, versions, and resource health. Once I identify the first point of failure, I fix that root cause and rerun only the required stage. After recovery, I verify the complete deployment and application health.
Simple ga gurthupettuko:
End-to-end failure → Git → CI/CD → Docker → Registry → Helm → Kubernetes → Application flow trace cheyyi → first failure point identify → fix → required stage rerun → health verify.
Eng: For an end-to-end deployment failure, trace the flow from Git through CI/CD, image/artifact, registry, deployment tooling, Kubernetes, and the application to find the first failure point.
Example:
git log -5 --oneline → Expected commit/code change identify cheyyi.
Jenkins/GitHub Actions logs → Build/test stage successful aa, exact failure ekkada undo chudu.
docker images → Expected image build ayyinda verify cheyyi.
docker inspect <image> → Image metadata/tag verify cheyyi.
docker pull <registry>/<image>:<tag> → Registry nunchi image accessible ga undho test cheyyi.
helm get values <release-name> -n <namespace> → Deployment values correct ga unnaya chudu.
helm get manifest <release-name> -n <namespace> → Kubernetes ki generated manifest verify cheyyi.
kubectl get pods -n <namespace> → Deployment Pods healthy ga unnaya check cheyyi.
kubectl describe pod <pod-name> -n <namespace> → Kubernetes events and failure reason chudu.
kubectl logs <pod-name> -n <namespace> → Application-side error identify cheyyi.
First failure point identify ayyaka → root cause fix chesi → required CI/CD/Helm/Kubernetes stage rerun cheyyi → kubectl rollout status deployment/<deployment-name> -n <namespace> → Deployment successful ga complete ayyindo verify cheyyi.

---

1. **41. Kubernetes Pod is running, but requests are intermittently failing. What will you check?**

Answer:
First, I check whether the issue is affecting all Pods or only specific Pods. Then I check Pod logs, readiness/liveness probe status, resource usage, and application response time.
I also verify Service endpoints, load balancing, connection limits, and whether one Pod is unhealthy or overloaded.
Based on the findings, I fix the unhealthy Pod, resource, application, or networking issue and verify that requests are stable.
Simple ga gurthupettuko:
Requests sometimes fail → affected Pods identify cheyyi → logs/probes/resources check cheyyi → Service endpoints and traffic distribution verify cheyyi → root cause fix chesi requests verify cheyyi.
Eng: Intermittent failures → identify affected Pods → check logs, probes, resources, and Service endpoints → fix the root cause → verify stable traffic.
Example:
kubectl get pods -o wide → Which Pods are running and on which nodes identify cheyyi.
kubectl describe pod <pod> → Probe failures and Pod events chudu.
kubectl logs <pod> --tail=100 → Application errors chudu.
kubectl top pod → CPU and memory usage check cheyyi.
kubectl get endpoints <service> → Service ki healthy Pod endpoints unnaya verify cheyyi.
kubectl get pods --show-labels → Service selectors correct Pods ni target chestunnaya check cheyyi.
Issue unhealthy Pod/resource/probe ayithe → fix chesi → kubectl rollout restart deployment <deployment> → traffic stable ga undho verify cheyyi.

---

42. Kubernetes node is Ready, but Pods are not getting scheduled on it. What will you check?
Answer:
First, I check the node conditions, taints, labels, and allocatable resources. Then I inspect the pending Pod events to understand why the scheduler is avoiding the node.
I also check node selectors, affinity rules, tolerations, and resource requests.
After identifying the scheduling restriction, I correct the Pod specification or node configuration and verify that the Pod gets scheduled.

Simple ga gurthupettuko:
Node Ready unna Pod schedule kakapothe → node taints/labels/resources check cheyyi → Pod events chudu → selector/affinity/toleration verify cheyyi → restriction fix cheyyi.
Eng: Node Ready but Pod is not scheduled → check taints, labels, resources, affinity, tolerations, and scheduler events → fix the restriction.
Example:
kubectl get nodes → Node Ready status chudu.
kubectl describe node <node> → Taints and allocatable resources check cheyyi.
kubectl describe pod <pod> → FailedScheduling reason chudu.
kubectl get pod <pod> -o yaml → NodeSelector, affinity, and resource requests verify cheyyi.
kubectl get nodes --show-labels → Required node label undho check cheyyi.
Issue taint/selector/resource ayithe → configuration correct chesi → kubectl get pods -o wide → Pod correct node lo schedule ayyindo verify cheyyi.

---

43. Kubernetes Service has endpoints, but traffic is still not reaching the application. What will you check?
Answer:
First, I verify that the Service selector matches the intended Pods and that the targetPort matches the application's listening port.
Then I test connectivity from inside the cluster and check NetworkPolicies, application binding, and container ports.
I also verify whether the application is listening on the expected interface and port.
After fixing the networking or application configuration, I test the Service again.
Simple ga gurthupettuko:
Endpoints unnayi but traffic ledu → selector/targetPort check cheyyi → Pod port and application listening check cheyyi → NetworkPolicy verify cheyyi → connectivity test cheyyi.
Eng: Endpoints exist but traffic fails → verify selectors, targetPort, application port, NetworkPolicy, and internal connectivity.
Example:
kubectl get svc <service> -o yaml → Selector and targetPort check cheyyi.
kubectl get endpoints <service> → Backend Pod IPs unnaya chudu.
kubectl exec -it <pod> -- curl http://<service>:<port> → Cluster internal connectivity test cheyyi.
kubectl exec -it <pod> -- ss -lntp → Application which port lo listen chestundo check cheyyi.
kubectl get networkpolicy -A → Traffic ni NetworkPolicy block chestunda verify cheyyi.
Issue targetPort/application binding ayithe → fix chesi → kubectl exec ... curl ... → Service response verify cheyyi.

---

44. Docker container cannot resolve DNS names. What will you check?
Answer:
First, I test DNS resolution inside the container and compare it with the host.
Then I check the container's /etc/resolv.conf, Docker DNS configuration, network settings, and host DNS availability.
I also verify whether the issue affects one container or all containers.
Based on the scope, I fix the Docker network or DNS configuration and retest name resolution.
Simple ga gurthupettuko:
Container DNS fail → container lo DNS test cheyyi → /etc/resolv.conf chudu → Docker network/host DNS check cheyyi → fix and retest.
Eng: Container DNS failure → test DNS inside the container → check resolver configuration and Docker networking → fix DNS → retest.
Example:
docker exec <container> cat /etc/resolv.conf → Container DNS server configuration chudu.
docker exec <container> getent hosts google.com → DNS resolution test cheyyi.
docker exec <container> ping <hostname> → Name resolution/network test cheyyi.
docker network inspect <network> → Container network configuration check cheyyi.
cat /etc/resolv.conf → Host DNS configuration compare cheyyi.
Issue DNS/network ayithe → correct DNS configuration apply chesi → docker exec <container> getent hosts <hostname> → resolution verify cheyyi.

---

45. Linux server is reachable by SSH, but application port is not reachable. What will you check?
Answer:
First, I verify whether the application process is running and listening on the expected port.
Then I check local firewall rules, security groups, network ACLs, and whether the application is bound only to localhost.
I also test the port locally and remotely to identify where the connection is being blocked.
After fixing the application binding or network restriction, I test the port again.
Simple ga gurthupettuko:
SSH work but application port fail → application listen chestunda chudu → local firewall/security group check cheyyi → localhost binding check cheyyi → remote connectivity test cheyyi.
Eng: SSH works but application port fails → verify the listening process, binding, firewall, security group, and remote connectivity.
Example:
ss -lntp | grep :8080 → Port 8080 lo application listen chestunda chudu.
curl http://localhost:8080 → Server local application response test cheyyi.
sudo ufw status → Local firewall status check cheyyi.
sudo iptables -L -n → Firewall rules inspect cheyyi.
nc -vz <server-ip> 8080 → Remote machine nunchi port connectivity test cheyyi.
Issue firewall/binding ayithe → rule or application binding fix chesi → nc -vz <server-ip> 8080 → port accessible ga undho verify cheyyi.

46. CI pipeline is successful, but the artifact uploaded to the repository is corrupted. What will you check?
Answer:
First, I verify the artifact generated during the build before uploading it.
Then I check the packaging command, file size, checksum, repository upload logs, and whether the upload completed successfully.
I also compare the local artifact with the downloaded artifact from the repository.
After identifying the packaging or upload problem, I fix it and publish a new verified artifact.
Simple ga gurthupettuko:
Build success but artifact corrupt → build artifact verify cheyyi → package/upload logs chudu → checksum compare cheyyi → correct artifact republish cheyyi.
Eng: Successful build but corrupted artifact → verify the generated file, upload process, checksum, and downloaded artifact.
Example:
ls -lh target/ → Generated artifact size check cheyyi.
sha256sum target/app.jar → Original artifact checksum generate cheyyi.
curl -O <artifact-url> → Repository nunchi artifact download cheyyi.
sha256sum app.jar → Downloaded artifact checksum compare cheyyi.
unzip -t <artifact.zip> → Archive corruption check cheyyi.
Issue packaging/upload ayithe → pipeline fix chesi → artifact malli publish chesi → checksum and download verify cheyyi.

---

47. Terraform plan shows unexpected infrastructure changes. What will you investigate?
Answer:
First, I review the Terraform plan carefully to identify which resources and attributes are changing.
Then I compare the current Terraform configuration, variables, state, provider version, and actual infrastructure.
I also check whether someone changed the infrastructure manually outside Terraform.
I do not apply the plan until I understand why the unexpected change is appearing.
Simple ga gurthupettuko:
Unexpected Terraform change → plan detail chudu → code/variables/state compare cheyyi → manual change/provider difference check cheyyi → reason clear ayyaka matrame apply cheyyi.
Eng: Unexpected Terraform changes → inspect the plan → compare configuration, variables, state, and actual infrastructure → understand the drift before applying.
Example:
terraform plan → Unexpected resources/attributes identify cheyyi.
terraform show → Current Terraform state details chudu.
terraform state list → State lo managed resources verify cheyyi.
terraform state show <resource> → Specific resource state compare cheyyi.
terraform providers → Provider versions check cheyyi.
git diff → Recent Terraform code changes identify cheyyi.
Manual change ayithe → actual infrastructure and Terraform configuration reconcile chesi → terraform plan → unexpected changes resolve ayyaya verify cheyyi.

---

48. Ansible reports changed successfully, but the expected configuration is still missing. What will you check?
Answer:
First, I verify the task output and confirm exactly what Ansible changed.
Then I check the destination path, file permissions, variables, privilege escalation, and whether the task executed on the intended host.
I also verify whether another process or configuration management tool is overwriting the change.
After identifying the reason, I correct the task and verify the final configuration on the target server.
Simple ga gurthupettuko:
Ansible changed ani chupinchina config lekapothe → correct host/path check cheyyi → variables/permissions/become verify cheyyi → overwrite avtunda chudu → final config verify cheyyi.
Eng: Ansible reports a change but configuration is missing → verify the target host, path, variables, permissions, privilege escalation, and possible overwrites.
Example:
ansible-playbook site.yml -l <host> -vvv → Task execution details chudu.
ansible <host> -m shell -a "cat /path/config" → Target config actual ga undho verify cheyyi.
ansible <host> -m stat -a "path=/path/config" → File existence and permissions check cheyyi.
ansible-inventory --host <host> → Host-specific variables verify cheyyi.
ps -ef | grep <process> → Configuration ni overwrite chestunna process unda check cheyyi.
Issue path/permission/variable ayithe → playbook fix chesi → ansible-playbook site.yml -l <host> → final configuration verify cheyyi.

---

49. Monitoring shows high application latency, but CPU and memory are normal. What will you check?
Answer:
First, I check whether the latency is coming from the application itself or from a downstream dependency.
Then I investigate database response time, external API calls, network latency, connection pools, thread pools, disk I/O, and application logs.
I compare the latency with previous healthy periods and identify when the problem started.
After finding the slow dependency or bottleneck, I fix it and monitor latency again.
Simple ga gurthupettuko:
Latency high but CPU/memory normal → application logs chudu → DB/API/network/connection pools check cheyyi → slow dependency identify cheyyi → fix and monitor.
Eng: High latency with normal CPU and memory → investigate application logs, database, APIs, network, connection pools, and other downstream dependencies.

Example:
curl -w "%{time_total}\n" http://<service> → Request response time measure cheyyi.
ss -s → Network connection state check cheyyi.
top → CPU/memory normal ani confirm cheyyi.
iostat -xz 1 5 → Disk I/O latency check cheyyi.
Application logs → Slow requests or timeout errors chudu.
Database monitoring → Slow queries and connection usage check cheyyi.
Issue DB/API/network bottleneck ayithe → root cause fix chesi → monitoring dashboard lo latency normal ki vachindo verify cheyyi.

---

50. Production application is down and multiple DevOps components may be involved. How will you troubleshoot it?
Answer:
First, I assess the impact and identify whether the issue is application-wide or limited to a specific environment, region, or component.
Then I follow the request path from DNS or load balancer to Ingress, Service, Pod, application, database, and external dependencies.
I check recent deployments, infrastructure changes, logs, metrics, events, and alerts to identify the first point of failure.
I prioritize restoring service safely, using rollback or another recovery action when appropriate, and then perform root-cause analysis after service recovery.
Simple ga gurthupettuko:
Production down → impact assess cheyyi → traffic path trace cheyyi → DNS/LB/Ingress/Service/Pod/App/DB check cheyyi → recent changes compare cheyyi → service restore cheyyi → root cause identify cheyyi.
Eng: Production outage → assess impact → trace the request path → check infrastructure, Kubernetes, application, and dependencies → identify recent changes → restore service safely → perform RCA.
Example:
dig <domain> → DNS resolution correct ga undho chudu.
curl -I https://<domain> → External endpoint response check cheyyi.
kubectl get ingress,svc,pods -A → Ingress, Services, and Pods status check cheyyi.
kubectl get events -A --sort-by=.lastTimestamp → Recent Kubernetes failures/events chudu.
kubectl logs <pod> --tail=200 → Application errors identify cheyyi.
kubectl describe pod <pod> → Probe, scheduling, image, and runtime issues check cheyyi.
git log -10 --oneline → Recent code changes identify cheyyi.
helm history <release> → Recent Helm deployments and revisions check cheyyi.
kubectl rollout history deployment/<deployment> → Deployment rollout history verify cheyyi.
Issue recent deployment ayithe → last known-good version ki controlled rollback chesi → kubectl rollout status deployment/<deployment> → application health verify cheyyi.
Service restore ayyaka → logs, metrics, deployment history, and infrastructure changes analyze chesi → permanent root cause fix cheyyi.